Building a Private Second Brain

What Breaks When You Keep AI Local, and Why That's the Point

A case study on designing a personal knowledge system with local AI models

Written for a general and executive audience

ABSTRACT

Organizations increasingly ask a simple question with a complicated answer: can we get the benefits of AI-assisted knowledge work — search, summarization, tagging, organization — without sending sensitive documents to a third-party service? This paper documents a working answer, built and tested over several weeks: a “second brain” system that watches a personal note-taking vault, ingests documents in common office formats, and makes them semantically searchable — using only AI models that run entirely on a single laptop, with no cloud dependency for processing. The system works, but it is the ways it *failed* during development that carry the more durable lessons. We report seven specific failure modes — a local vision model that hallucinated on business diagrams, a language model that invented file paths, ambiguous identical filenames, a file-architecture choice that quietly doubled cognitive load, uncontrolled document-version sprawl, and a false assumption about where data physically lived — and generalize each into a recommendation for anyone evaluating or deploying local AI tooling in a business context. The accompanying source code is published for reproducibility.

1. Introduction

“Second brain” is the popular name for a simple idea: instead of trusting memory to hold every note, meeting summary, and reference document, keep them in one searchable place and let a system help retrieve and connect them. Tools like Obsidian, Notion, and Roam Research turned this from a personal habit into a category of software over the past decade. The natural next step — and the one every vendor in this space is now taking — is to point a large language model at that pile of notes so it can search, summarize, and organize on your behalf.

That step usually means sending your notes to a cloud API. For a hobbyist's reading list, that may be an acceptable trade. For a consultant's client materials, a lawyer's case files, or an executive's strategy notes, it often is not — the documents are exactly the ones an organization is most careful about, and “we uploaded it to a third-party AI vendor for indexing” is not a sentence most compliance functions want to sign off on lightly.

This paper describes a system built to test whether that trade-off is actually necessary: whether the useful parts of "AI-assisted second brain" — semantic search, automatic tagging, cross-document linking, daily summaries — can be delivered by models that never leave the machine they run on, using Ollama, a tool for running open-weight AI models locally, paired with a local vector database for search. The system was built, used, and iterated on with real (if redacted, for this paper) documents over several weeks, and is published in full at the accompanying GitHub repository (see "Reproducibility").

We structure this paper around a case-study format because the interesting content is not the architecture diagram — it is mundane by design — but the sequence of things that did not work on the first attempt, and what each failure implies for anyone building or buying similar systems.

2. Related Work

This project sits at the intersection of two active threads, and takes a position relative to each. The first is the "LLM wiki" pattern popularized by Andrej Karpathy in 2026: rather than re-deriving answers from scratch on every query, an agent compiles source material once into a maintained markdown knowledge base — several Obsidian-specific forks of that pattern already exist. The system in this paper is a variant of the same idea, but makes a different bet about where the engineering effort belongs. The wiki-pattern projects mostly orchestrate a capable coding agent to write and rewrite pages on request; this system instead runs a small, always-on watcher that reacts to filesystem events and applies deterministic policy — deduplication, format preference, link disambiguation — without invoking a model at all for most of that work. We think this is the more defensible design for something that runs unattended: a background process that only calls a language model for the two tasks language models are actually good at — summarizing and judging relevance — is easier to reason about than one that also asks an agent to make and remember structural decisions.

The second thread is local retrieval-augmented generation for personal notes, which already has working, shipped tools — Reor and ObsidianRAG being the closest to this project in intent, both pairing Ollama with a local vector store for offline question-answering over a note collection. Where this paper differs is emphasis. Those projects optimize the query experience — asking a good question and getting a good answer — and treat ingestion as a solved preliminary step. This paper's position, argued through Section 5, is closer to the opposite: for a real, multi-year personal or organizational document collection, ingestion is where nearly all of the actual engineering difficulty lives — version sprawl, format duplication, naming collisions, cloud-sync side effects — while query-time RAG mechanics are comparatively well understood and rarely where things went wrong in practice.

Finally, the specific failure described in Section 5.2 — a language model asked to reproduce an exact reference inventing one instead — is not particular to the small model used here. Recent large-scale studies of LLM-generated citations report similar failure rates in far larger, commercially-deployed models: one legal-domain study measured 13–21% citation hallucination even with retrieval grounding in place, and a separate large-scale analysis of structured citation output found only about half of extracted references were valid. Read against this paper's experience, the pattern looks less like an idiosyncrasy of small local models and more like a general property of generative text models used as a source of exact, structured fact — which is the basis for this paper's recommendation to keep models out of that role entirely, rather than trying to prompt the failure rate down.

3. Why Keep It Local?

Three considerations, in order of how often they came up in practice:

- **Privacy by construction, not by policy.** A privacy *policy* is a promise about what a vendor will and won't do with your data. A privacy *architecture* removes the vendor from the loop entirely. When embeddings, search, tagging, and summarization all run on local models, there is no API call carrying document content to audit, no data-processing agreement to negotiate, and no vendor retention window to worry about. The trust boundary collapses to the laptop itself.
- **Cost that doesn't scale with usage.** A system that re-indexes a growing archive and re-evaluates it continuously — which is what “always up to date” requires — makes a meaningful number of AI calls per day, indefinitely. Metered cloud APIs turn that into a running bill that grows with exactly the behavior you want to encourage (using the tool more). Local models, once downloaded, are effectively free to run; the cost is paid once, in hardware, not per query forever.
- **No external dependency for uptime or rate limits.** The system works on a train, in a client's basement server room with no signal, or during a provider outage. This matters more for an executive's daily habit-forming tool than it sounds: a system that occasionally doesn't work gets abandoned.

The honest trade-off is capability and speed: local models, especially on a laptop without a dedicated GPU, are smaller and slower than the frontier cloud models. Section 5 treats that trade-off as the central design constraint, not a footnote.

4. What the System Does

At a high level, the system has four moving parts:

1. **Ingestion.** Documents (PDF reports, Word documents, PowerPoint decks, plain text) are dropped into a folder, or pointed to in an external location such as a shared drive. The system extracts the readable content — headings, bullet points, tables, speaker notes in a slide deck — into a structured note.
2. **Indexing.** Each note is broken into passages, and each passage is converted into a numerical representation (an “embedding”) by a small local model. These embeddings are stored in a local vector database, which is what makes search “semantic” — it can find a passage about “budget overruns” when you search for “we're spending too much,” because the two mean similar things to the model, not because they share keywords.
3. **Organization.** A second, slightly larger local model reads each new note alongside the notes it's most semantically similar to, and proposes short tags and links between related material — the connective tissue a second brain is supposed to provide.
4. **Retrieval.** A search box (available from the command line and from inside the note-taking application) turns a plain-language question into an embedding, finds the closest matching passages, and returns them.

None of this requires a network connection once the models are downloaded. Figure 1 sketches the flow.

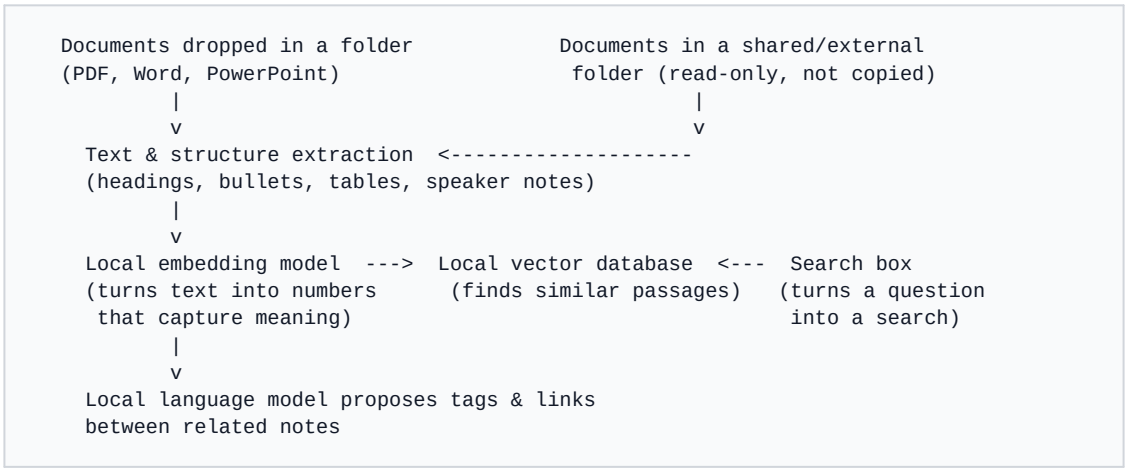

Figure 1. System flow (simplified).

5. A Case Study in What Broke

Each of the following was a working feature that was removed, replaced, or redesigned after real use exposed a problem. We include them because the failure is more generalizable than the eventual fix.

5.1 A local vision model confidently hallucinated on business diagrams

An early version of the system extracted the images embedded in slide decks — architecture diagrams, org charts, screenshots — and used a small local vision model to write a one-sentence caption, so the diagram's content would become searchable text. Two things went wrong. First, the model produced empty output whenever asked to write in a language other than English, without any error — it simply returned nothing, silently. Second, and more seriously, when it did produce a caption for a genuine business diagram, it sometimes invented a plausible-sounding but entirely wrong description — one diagram of a technical operating model was captioned as “a close-up view of a group of nerves, each with a bright orange hue... connected to the brain.”

The lesson generalizes beyond this one model: **a wrong answer stated with total confidence is more dangerous than no answer at all**, because a wrong caption gets stored and treated as fact by everything downstream — including future searches that will surface it. The feature was removed rather than shipped as “mostly working.” Where an AI capability's failure mode is silent and confident, the responsible default is often not to deploy it at that model size or task, not to add a disclaimer and ship it anyway.

5.2 Never let a generative model be the source of truth for a fact that must be exactly correct

A related but distinct issue appeared in the tag-and-link feature. The system originally asked the language model to write out the actual link text pointing to a related note, as free-form output. Language models are good at generating plausible text; they are not reliable at reproducing an exact string — such as a file path — byte for byte. In practice, the model occasionally invented a path that

didn't exist, or spliced together fragments of two real paths into something that looked correct but pointed nowhere.

The fix illustrates a pattern with much broader applicability: **ask the model to choose from a numbered list, and have ordinary code construct the actual fact from the chosen option.** The model is good at judgment ("which of these five things is most relevant?"); the code is good at precision ("write out this exact string"). Mixing the two — asking the model to both judge *and* transcribe — is where the reliability problem showed up. This division of labor is a cheap, general-purpose safeguard for any system where an LLM's output will be used as a reference, a citation, or a piece of structured data rather than prose to be read by a human.

5.3 Two real things can share the same name — and systems built on names, not identities, will get it wrong

The document collection this system indexed included two genuinely different files that happened to have an identical filename, stored in two different folders (a common occurrence when different teams reuse a template's default filename). A short reference to "the file named X" was, in that situation, ambiguous by construction — there were two different, correct answers.

This is a data-hygiene problem, not an AI problem, and it would break any system — AI-powered or not — that identifies content by name rather than by a stable, unique identity. The system now detects this specific collision and automatically falls back to a fully qualified reference only in the cases where it's actually needed, which keeps ordinary references short while still being correct in the rare ambiguous case. The broader point for any organization building on top of existing file collections: **naming collisions are common enough in real document repositories that "assume unique filenames" is not a safe assumption**, and detecting the exception is cheaper than requiring everyone to rename things.

5.4 A safety-motivated design choice quietly doubled the system's complexity

To avoid ever silently modifying content a person had written by hand, the earliest version of the tagging feature wrote its suggestions into a companion file, mirroring the exact folder structure of the original notes. This was the right precaution for hand-written notes. But it was applied uniformly — including to notes the system itself had generated from source documents, where there was no hand-written content to protect in the first place.

The practical effect: every single document produced two files, in two parallel folder trees, and understanding one document meant looking in two places. This is a common shape of over-engineering: a safety rule that is correct for one category of content gets applied to all content, and the cost (in this case, roughly doubling the number of files and folders a person has to navigate) is paid indiscriminately. The fix was to distinguish the two cases explicitly — content the system generated gets enriched in place; content a person wrote by hand stays untouched, exactly as before — which cut the visible file count roughly in half without weakening the original safety property at all. The general lesson: **a safeguard's scope should match the actual risk it addresses, not be applied blanket-wide for simplicity**, because the "simple" blanket rule can itself become the source of complexity.

5.5 Duplicate documents need an explicit, automatic policy — not reliance on people remembering to clean up

Any document collection that multiple people touch accumulates near-duplicates: `report_v1.pptx`, `report_v3.pptx`, `report_vFINAL.pptx`, and often a lighter PDF export of the same final deck sitting alongside it. Indexing all of them means the search results are cluttered with outdated drafts, and a search may just as easily surface the wrong version as the right one.

Two simple, mechanical rules turned out to resolve nearly every real instance of this: keep only the highest version number (with an explicit "FINAL" marker always outranking any number), and — separately — when the same document exists as both a PDF and an editable Office file, keep only the PDF, since it is almost always dramatically smaller for identical content. Neither rule requires judgment or an AI model; both are pure bookkeeping, applied automatically every time a new file appears. The lesson for any AI-adjacent system built on a real document archive: **most of the "garbage in, garbage out" problem in practice is not exotic — it's version sprawl and format duplication, and it responds well to simple, deterministic policy, not to more AI.**

5.6 A claim about data locality needs to be verified technically, not assumed from configuration

Partway through this project, a routine check of the note-taking folder's actual location on disk revealed that it was not a plain local folder at all: the operating system's own "back up your folders to the cloud" feature had transparently redirected it into a cloud-storage sync folder, months earlier, for reasons unrelated to this project. Every file the system had written — including notes generated from confidential source material — had been silently synchronizing to a cloud account the entire time.

The AI *processing* had, in fact, been entirely local, exactly as designed. But the *storage* had not been, and those are two different guarantees that are easy to conflate in a casual description like "nothing leaves the machine." For any organization evaluating a "local AI" or "on-device AI" claim — whether built in-house or purchased from a vendor — this is worth stating plainly: **verify where the files actually live on disk, don't infer it from where you told the software to put them.** Modern operating systems and productivity suites increasingly redirect standard folders into cloud sync locations by default, often invisibly to the end user, which means a privacy architecture can be entirely correct in its AI design and still leak through the filesystem underneath it.

5.7 Real-time reactivity is not always the right default — especially for content you don't own

The system initially treated every folder it watched identically: check continuously, react within seconds. For a folder someone else owns and edits — a shared drive, a colleague's export — this is both unnecessary (a shared document doesn't need to be searchable within two seconds of a coworker saving it) and mildly risky (continuously reading from a location outside your control invites edge cases, like reading a file mid-save).

The system now treats externally-owned content differently by default: check once a day, read-only, and never delete the local record even if the source file is later removed upstream — the local copy becomes a deliberate, conservative archive rather than a live mirror. The general principle: **reactivity**

should be proportional to ownership and volatility, not maximized by default; a calmer, periodic, read-only posture is often the more appropriate one for data a system doesn't control.

6. What This Suggests for Organizations Evaluating Local AI

Pulling back from the specific system, four broader implications seem worth stating for a reader deciding whether “run it locally” is a realistic option for their own organization, rather than a purist's preference:

- **It is genuinely feasible today, on ordinary hardware, for text-centric tasks.** Search, tagging, summarization, and light organization ran acceptably on a single laptop with no dedicated graphics hardware. This is a meaningfully lower bar than “we need a GPU cluster,” and it changes the calculus for smaller teams and individual professionals, not just large IT organizations.
- **Local AI capability is currently uneven across tasks, not uniformly “behind” cloud AI.** Text generation and understanding, for models of a size that runs comfortably on a laptop, were reliable enough to build on. Image understanding, at that same size class, was not — the gap there is larger than the gap in text. Any evaluation of “local AI” should test the specific task in question rather than treating “AI” as one capability that is either ready or not.
- **The riskiest failures were quiet, not loud.** None of the problems in Section 5 caused a crash or an error message. They produced plausible-looking wrong answers, or a folder structure that was merely annoying rather than broken. This is the general shape of AI-adoption risk worth planning for: not the system going down, but the system confidently doing the wrong thing in a way nobody notices until later.
- **Data-locality claims are a technical fact to verify, not a policy statement to trust.** Section 5.6's discovery generalizes directly to any procurement conversation about “on-premise” or “local” AI tooling: ask how to verify it, not just how it's described.

7. Limitations and Future Work

This is a single-user prototype, developed and tested by one person over several weeks against their own document collection, not a hardened multi-user product. Several caveats follow directly from that: the reliability observations throughout are anecdotal, not benchmarked; hardware was a single consumer laptop without a dedicated GPU, and performance (especially for anything beyond short text generation) would look different on stronger hardware; and the local vision-model gap described in Section 5.1 is a fast-moving area — it is entirely plausible that a system built six months later would find that particular limitation already narrowed. The intent of this paper is not to claim these specific numbers or model choices are permanent, but to document a set of failure *shapes* — silent hallucination, model-as-transcriber, identity-by-name, safeguard-overreach, uncontrolled duplication, unverified locality, reactivity mismatch — that are likely to recur in any similar system regardless of which specific models are used to build it.

8. Conclusion

The technically interesting part of this project was not that a local, private “second brain” is possible — it clearly is, and the components to build one are freely available today. The more useful part, for anyone building or evaluating a similar system, is the specific list of ways it went wrong first: a model that lied confidently, a model asked to do a job better suited to ordinary code, two files with one name, a safeguard applied too broadly, clutter with no policy to contain it, and a privacy claim that turned out to need verifying rather than assuming. None of these are exotic AI failures; all of them are the kind of thing that shows up the first time a system meets real, messy, human data. Building for that reality — rather than for the clean demo — is most of the actual work.

Reproducibility

The complete source code for the system described in this paper — the document ingestion pipeline, the deduplication logic, the note-taking application plugin, and the daily digest generator — is published under an open-source license at:

github.com/alebellotta/local-second-brain

The repository intentionally omits any actual document content, personal file paths, or organization-specific configuration; it is meant to be read and adapted, not run unmodified against someone else's files.

References

[1] Karpathy, A. (2026). *LLM Knowledge Bases* [gist]. gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

[2] *Reor* — private, local AI personal knowledge management app [software]. github.com/reorproject/reor

[3] *ObsidianRAG* — privacy-first RAG for Obsidian notes using local AI [software]. github.com/Vasallo94/ObsidianRAG

[4] Citation Grounding: Detecting and Reducing LLM Citation Hallucinations via Legal Citation Graphs (2026). *arXiv:2606.00898*. arxiv.org/pdf/2606.00898

[5] GhostCite: A Large-Scale Analysis of Citation Validity in the Age of Large Language Models (2026). *arXiv:2602.06718*. arxiv.org/pdf/2602.06718

[6] Gao, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey (2023/2026). *arXiv:2312.10997*. arxiv.org/abs/2312.10997